

Monitoring screenshots

Milestone 3 evidence: "Ops runbook (deploy/rollback/incident) in repo + **monitoring screenshots**". The three monitoring services — Sentry (errors), Grafana Cloud / Tempo (traces and metrics), PostHog (web analytics, no PII) — were instrumented in code (`apps/api/src/instrumentation.ts`, `apps/web/src/lib/observability.ts`) and verified live in production on 2026-09-08. The screenshots were captured on 2026-09-11, directly against the live dashboards.

No staged data. Every screenshot below shows what each service reported at capture time, unedited — real traffic and real errors from the production instance (`propnexus-api.onrender.com` / `propnexus-web.onrender.com`), not a mock or a seeded demo state.

How this also covers Output 4's telemetry

Output 4 (Testing & Security) asks for "telemetry for coverage/latency/error budgets". The same instrumentation shown on this page provides two of the three, and the test report provides the third:

Telemetry	Where it comes from	Evidence
Coverage	Vitest coverage on every CI run; all four TypeScript parts have minimum thresholds that fail the build	The coverage table in the test report (100% of lines)
Latency	OpenTelemetry traces of every API request, exported to Grafana Cloud (Tempo), with per-middleware timing	<code>grafana-tempo-trace-detail.jpg</code> below
Error budgets	The target below, measured on the OpenTelemetry traces of every API request in Grafana Cloud (Tempo); Sentry captures each of those errors with its stack trace	The target, indicator and measurement below, <code>sentry-issues.jpg</code>

The reservation → escrow latency that acceptance criterion 3 measures (median under 12 minutes) is a separate, product-level metric, served by the API at `GET /api/v1/audit-logs/telemetry/reservation-to-escrow` and reported under evidence item 3.

The error budget

Target (SLO)	99% of API requests answered without a 5xx, over a rolling 30-day window. The budget is the remaining 1%
Indicator (SLI)	Server requests answered with <code>http.response.status_code ≥ 500</code> over all server requests, counted on the root server span of each trace in Tempo (<code>{ resource.service.name = "propnexus-api" && span.kind = server && nestedSetParent < 0 }</code> <code>count_over_time()</code>), and the same with <code>&& span.http.response.status_code ≥ 500</code>)
Why traces and not the metric	The API also exports the <code>http.server.request.duration</code> histogram, but on the free tier the instance sleeps after 15 minutes and every wake-up restarts its counters under the same series. A rare error then shows up as a flat counter, and <code>increase()</code> reports 0 errors where Tempo shows them. The traces count each request once

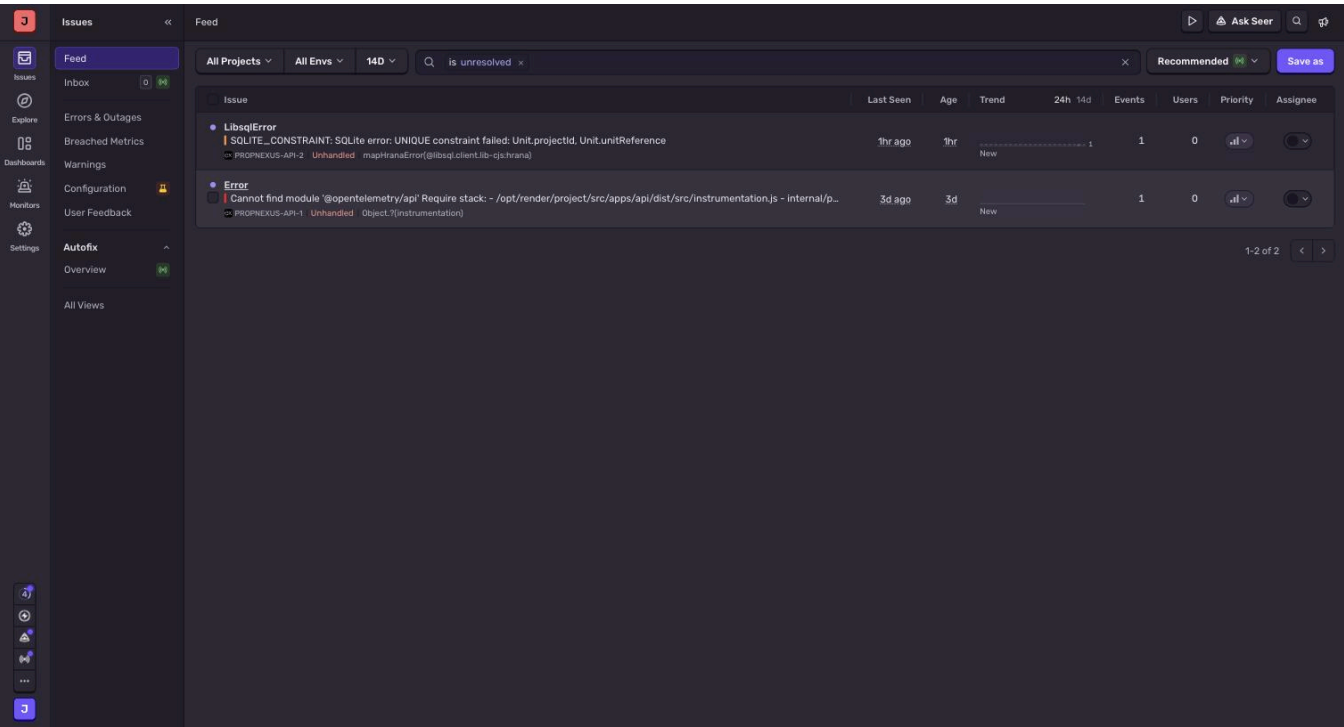
Measured on 2026-10-05 (30 days, 2026-09-05 → 2026-10-05)	138,796 requests, 22 with a 5xx: 99.98% , inside the 99% target. The 22: 6 × 500 and 6 × 503 without a matched route, 4 × 503 on the geocoding proxy, 3 × 500 on <code>POST /evidence/reconcile</code> , 2 × 503 on <code>/health</code> , 1 × 500 on unit creation. The three reconcile errors were a start-up race, found through this measurement and fixed the same day: after a cold start, the anchoring service took about 25 seconds longer than the HTTP server to be ready
Where each error is investigated	Sentry, which receives every 5xx with its stack trace (<code>sentry-issues.jpg</code>). 4xx responses are client errors and do not consume the budget
What it does not see	A request that never reaches the API process, such as one that fails while the free-tier instance is waking up (about two minutes after 15 minutes without traffic), is not in the metric. It is measured at the API, not at the edge

The target was set on 2026-10-05. There is no alerting on it yet: on Preprod, with pilot traffic, it is reviewed by hand, and alerting is part of the work before mainnet.

Files

All under [monitoring/](#).

`sentry-issues.jpg`

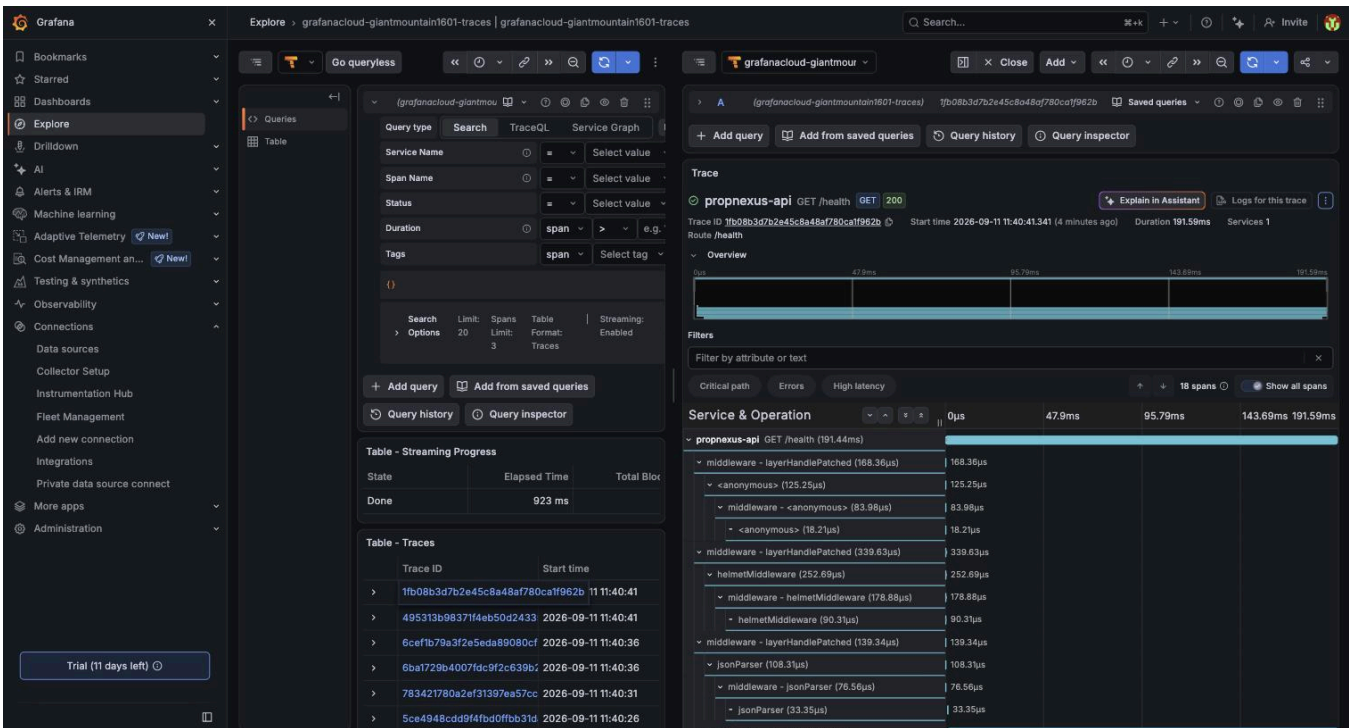


Sentry's issue feed for the `propnexus-api` project, filtered to unresolved issues. Two real, unhandled errors are visible:

- `LibsqlError – SQLITE_CONSTRAINT: UNIQUE constraint failed: Unit.projectId, Unit.unitReference`, 1 hour old at capture time. This one is incidental, not a monitoring artifact: it was produced during this same session's manual exercise of the reservation-to-escrow flow, submitting a unit-creation form twice against the same reference. Left in the screenshot on purpose, as further proof the pipeline captures real errors as they happen, not synthetic ones.
- `Error – Cannot find module '@opentelemetry/api'`, 3 days old — a historical, already-fixed incident whose issue entry is still open in Sentry because nobody has manually resolved it there; the

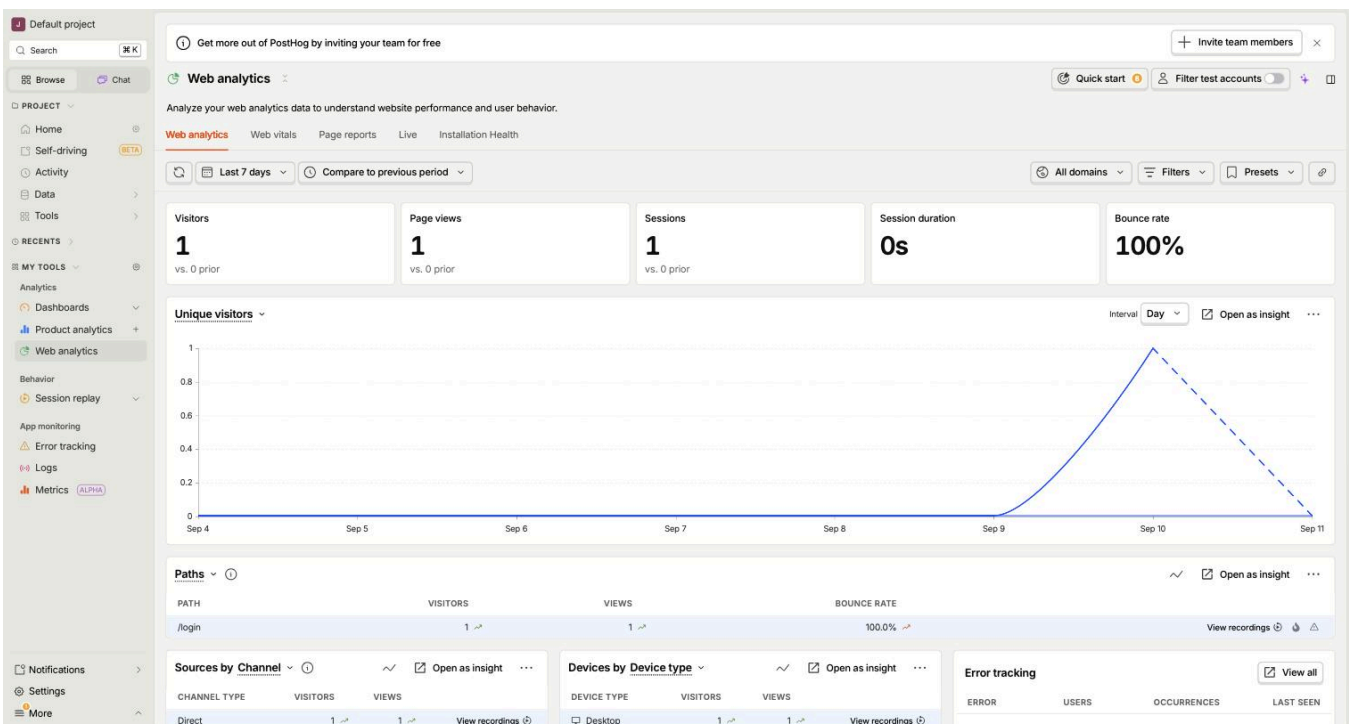
underlying bug has been fixed in code since.

grafana-tempo-trace-detail.jpg



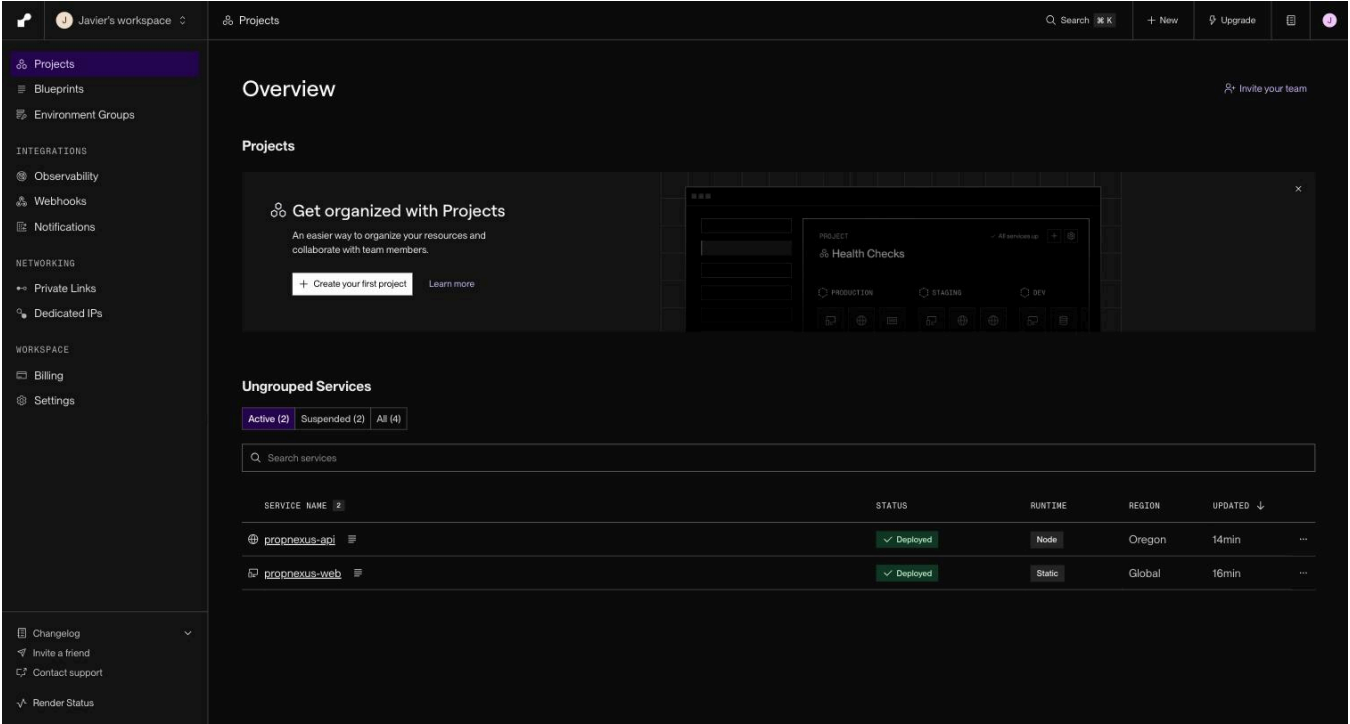
Grafana Cloud → Explore → the grafanacloud-giantmountain1601-traces (Tempo) datasource. Left panel: the trace search table over the last 24 hours, real propnex-api GET /health traces with real timestamps and durations. Right panel: one of those traces opened to its full waterfall — 18 spans, per-middleware timing breakdown (helmetMiddleware, jsonParser, etc.), 191.59 ms total duration, HTTP 200.

posthog-web-analytics.jpg



PostHog → Web analytics, last 7 days, for the `apps/web` production deployment. Shows a real visitor session against `/login` on 2026-09-10, with visitor/pageview/session counts, bounce rate, and the unique-visitors time series. (The dedicated "Web vitals" tab returned no data for the selected range — traffic volume so far is too low for that specific metric to have recorded; the general Web analytics view already demonstrates the pipeline is live and receiving real events, which is what the evidence asks for.)

render-services-deployed.jpg



Not required on its own (the public URL is covered elsewhere), included as supporting context: both Render services, `propnexus-api` and `propnexus-web`, `Deployed` at capture time.

Summary

Service	What it monitors	Status at capture
Sentry	Application errors (<code>apps/api</code>)	Live, 2 real unresolved issues
Grafana Cloud (Tempo)	Distributed traces (<code>apps/api</code>)	Live, real traces in the last 24h
PostHog	Web analytics (<code>apps/web</code>)	Live, real session recorded
Render	Deploy status (both services)	Deployed , both